Feature Flags Shipping Software Faster and Safer

What feature flags are, why they matter, a live demo, and real-world use cases.

A session for the University of Edinburgh students group · 20–30 min

David Sandilands
Staff Solutions Architect & EMEA Lead, LaunchDarkly

University of Edinburgh

LaunchDarkly

Me — From Edinburgh CS Grad to Solutions Architect



David Sandilands

- BEng (Hons) Computer Science, University of Edinburgh (2001–2005)
- 8 years at NatWest as a Cloud Infrastructure Engineer, building their IaaS platform in a major digital transformation
- Wore several hats at Puppet: Product Lead for developer tools and the Puppet ecosystem (Forge, PDK, integrations), then Community & DevRel Lead
- Authored Puppet 8 for DevOps Engineers (Packt); co-authored the State of DevOps / Platform Engineering Report
- Now Staff Solutions Architect & EMEA Lead at LaunchDarkly — helping engineering and product teams turn feature management into a strategic advantage
- Outside work: all 282 Scottish Munros (Munroist #3085), sci-fi & fantasy books, days out with my wife and two sons

Edinburgh
CS grad

NatWest
Cloud Infra Engineer

Puppet
Product Lead, Dev Tools → Community / DevRel Lead

LaunchDarkly
Staff SA / EMEA Lead

LaunchDarkly — The Runtime Control Plane for Modern Software

Decouple feature rollout from code deployment: test in production, release gradually, and manage flags across their lifecycle.

2014

Founded · HQ in Oakland, California

5,000+

Customers globally

20%+

of the Fortune 100

A feature management and experimentation platform — safely manage code and AI agents in production with feature flags, progressive delivery, automated rollback, and runtime control.

Enterprise-grade trust: SOC 2 Type II · ISO 27001 · FedRAMP · GDPR-compliant · No PII required

What is a Feature Flag?

A software technique to turn a feature on or off without changing source code or redeploying.

```
if (client.boolVariation("new-checkout",
ctx, false)) {
    renderNewCheckout();
} else {
    renderOldCheckout();
}
```

new-checkout ON

Flip instantly — no redeploy

Target precisely — by user, segment, region

Types: boolean · string · number · JSON

Not a config file. Config changes at deploy time and is on/off for everyone; flags flip at runtime and target precisely the foundation of a continuous release, observe, iterate loop.

Why Feature Flags? Separate Deploy From Release

Code can live in production without executing until you decide it's ready.

Deploy = Release

Risky big-bang launch

100% of users at once

If it breaks → full rollback or redeploy

Deploy ≠ Release

Controlled progressive rollout



If it breaks → flip off in seconds

Slash risk & MTTR (hours → minutes) · **Enable trunk-based development** · **Empower the whole team** (product, marketing, sales, support)

Use Cases — Where Feature Flags Shine



Release management
Canary releases, betas, and percentage rollouts to limit the blast radius.



Operational reliability
Kill switches and circuit breakers to disable a bad feature the moment alerts fire.



Targeting & personalization
Tailor experiences by user, plan, device, or region — and manage entitlements.



Experimentation & A/B
Run tests on live traffic to learn whether you built the right thing.



De-risk migrations & AI
Roll out infra/AI changes in safe chunks with instant rollback.

Live Demo — From a Simple Flag to Data-Driven Releases

Four steps, building up from a single boolean to guarded, experiment-driven rollouts.

1

A boolean flag

Create a true/false “hello world” flag, wrap it in code, and toggle it live — an instant change with no redeploy.

→

2

Targeting with contexts

Serve different variations to specific users, segments, or context kinds (user, device, org) by their attributes.

→

3

Progressive rollout

Move from all-or-nothing to a percentage rollout — 10% → 25% → 50% → 100% — to limit blast radius.

→

4

Guarded, experiments & AI

Feed live metrics so a Guarded Release auto-rolls back on a regression, and an A/B test shows which variation wins.

Instant change → precise targeting → limited blast radius → data decides

One flag picks the customer journey.

Challenge

Showing customers a different quote journey meant a database change, a database analyst and cross-team coordination.

Solution

Flags and segments pick the journey from backend brand data: Admiral only, Gladiator only, or both. Workflows schedule changes and approvals gate production.

Results

2 → 8
deployments a month

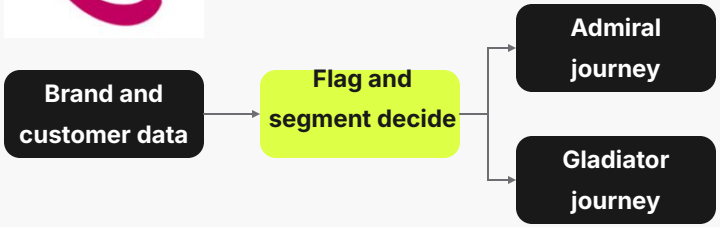
1 month → 1 week
lead time

Hours → seconds
time to recover (MTTR)

Below 15%
change failure rate

Admiral

UK insurer, Cardiff, FTSE 100



Any customer, one brand, or both. Simplified view.

"LaunchDarkly allows us to create targeted personalized experiences for our customers without requiring custom code."

Enrique Vicent Moratalla, Chapter Lead, Admiral Insurance

Move to Azure one slice at a time.

Challenge

Two releases a month, often rolled back, plus a multi-year move to .NET, Next.js and Azure too risky to cut over at once.

Solution

Azure app services run beside the legacy app. A flag routes a slice of users to the new path, and back if errors appear; the slice grows as Datadog APM shows it healthy.

Results at six months, with Datadog APM

On demand
deployments, from 2 a month

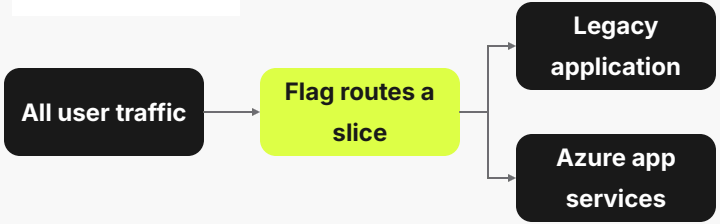
14 days → 3 hours
lead time

Days → under 1 hour
recovery time

75% → 8%
change failure rate

Hireology

US hiring platform, founded 2010



Healthy: widen. Errors: route back. Simplified view.

"What LaunchDarkly does is it provides a safety net for this change so we can test out the Azure app services without exposing it to everybody in the world."

Nick Macellaio, DevOps Manager, Hireology

LaunchDarkly

Questions?

Thank you let's talk feature flags, LaunchDarkly, and careers in software delivery.

Try it yourself: free trial and docs at launchdarkly.com

Connect: linkedin.com/in/davidsandilands



David Sandilands · Staff Solutions Architect & EMEA Lead